

VecForge 2.0

Complete Instructions: install, search, remember, serve.

Local-first | AES-256 encrypted | Hybrid dense + BM25 search | AI memory for Claude | MCP server | Quantum-inspired rerank | REST API & CLI

Built by **Suneel Bose K**
Powered by **ArcGX TechLabs Private Limited**

vecforge 2.0.0 · October 2026

Contents

1	What VecForge is	overview, architecture
2	Installation	pip, extras, offline mode
3	Quickstart	your first vault in 5 lines
4	Core concepts	vaults, namespaces, indexes
5	Adding and managing data	add, batch, upsert, ingest, delete
6	Searching	alpha, filters, rerank, quantum
7	AI memory	MemoryStore, sessions, context packs
8	Connecting Claude over MCP	Desktop, Code, HTTP
9	Embedding models	sentence-transformers, hash, any API
10	Security	encryption, RBAC, audit, backup
11	REST API	server and endpoints
12	Command line	every command
13	Multimodal	images and audio
14	Performance and tuning	benchmarks, index types
15	Upgrading from 1.x	breaking changes
16	Troubleshooting	common problems
17	Quick reference	one-page cheat sheet

1 What VecForge is

VecForge is a vector database that runs inside your Python process and stores everything in one file on your machine. It finds documents by meaning and by keyword at the same time, can encrypt the file. New in 2.0, it also gives AI assistants a long-term memory they can use across sessions.

Local-first
No server to run, no account, no API key. A vault is a single SQLite file you own.

Hybrid search
Semantic (FAISS) and keyword (BM25) results are fused, then optionally reranked.

AI memory + MCP
Claude Desktop, Claude Code and other MCP clients remember facts, preferences and decisions between conversations.

Enterprise basics
AES-256 encryption, namespaces per tenant, roles and API keys, an audit log and safe backups.

How a search works

1. **Dense retrieval.** Your query is turned into a vector and compared with every document vector in FAISS.
2. **Keyword retrieval.** VecForge's own BM25 inverted index scores documents that share words with the query.
3. **Fusion.** Both score lists are normalised to the range 0 to 1 and blended with `alpha` (0 = keywords only, 1 = meaning only).
4. **Hydration and filters.** Matching documents are loaded and metadata filters are applied. If a filter removes too many, VecForge automatically looks further down the list.
5. **Reranking (optional).** A cross-encoder model and/or Grover amplitude amplification reorder the top results.
6. **Recency (optional).** Newer documents can be nudged up.

Namespace limits and deletes are enforced inside both retrievers, so small namespaces always get complete results.

What lives in a vault file

Part	Purpose
documents	Text, embedding vector, metadata, namespace, modality, timestamps, content hash
namespaces	Tenant boundaries (`default` always exists)
index snapshot	Saved FAISS + BM25 indexes so the vault reopens without rebuilding
sessions, key/value	Memory sessions, internal counters, hashed API keys

2 Installation

VecForge needs Python 3.10 or newer. It is tested on Linux, macOS and Windows with Python 3.10 to 3.13.

Install from PyPI

```
# core library, CLI and REST server
pip install vecforge

# add the MCP server for Claude
pip install "vecforge[mcp]"

# several extras at once
pip install "vecforge[mcp,encryption]"
```

Optional extras

Extra	Adds	Use it for
`mcp`	mcp SDK	The memory server for Claude Desktop / Claude Code
`encryption`	sqlcipher3-binary	AES-256 encrypted vaults
`multimodal`	open-clip-torch, openai-whisper, Pillow	Image and audio search
`cloud`	boto3, GCS and Azure clients	Uploading encrypted backups
`gpu`	faiss-gpu, cupy	GPU acceleration
`quantum`	qiskit, qiskit-aer	Real quantum-circuit experiments (not needed for reranking)
`dev`	pytest, ruff, black, mypy, httpx, mcp	Contributing

Install from source

```
git clone https://github.com/bosekarmegam/vecforge.git
cd vecforge
pip install -e ".[dev]"
pytest tests/ -q
```

Offline or no-download mode

The default embedding model (`all-MiniLM-L6-v2`, about 90 MB) downloads once from Hugging Face. On an offline machine, in CI, or when you want instant start-up, use the built-in hashing embedder instead:

```
db = VecForge("vault.db", model_name="hash")
# or for every command and script:
export VECFORGE_MODEL=hash
```

> **What "hash" means**
>
> The hash embedder matches shared words and spellings, not meaning. It suits tests, keyword-heavy data and offline use. For semantic search, use a sentence-transformers model.

Platform notes

- **Windows:** PyTorch needs the Microsoft Visual C++ Redistributable (`aka.ms/vs/17/release/vc_redist.x64.exe`). Without it you see `WinError 126 … c10.dll`.
- **Linux:** `sudo apt-get install -y build-essential` if a wheel has to compile.
- **Encryption from source:** macOS `brew install sqlcipher`; Ubuntu `apt-get install libsqlcipher-dev`, then `pip install sqlcipher3`.

Check the installation

```
vecforge --version            # VecForge, version 2.0.0
python -c "import vecforge; print(vecforge.__version__)"
```

3 Quickstart

Your first vault

```
from vecforge import VecForge

db = VecForge("my_vault.db")
db.add("Patient admitted with type 2 diabetes", metadata={"ward": "7"})
results = db.search("diabetic patient")
print(results[0].text)
```

The file `my_vault.db` now holds the document. Open it again tomorrow with the same line and the index is restored from its snapshot.

A slightly bigger example

```
with VecForge("clinic.db") as db:          # saves and closes automatically
    db.add_batch(
        ["Type 2 diabetes, hip fracture admission",
         "Cardiac arrest, ICU transfer",
         "Routine diabetes follow-up"],
        metadata_list=[{"year": 2024}, {"year": 2024}, {"year": 2023}],
        namespace="ward_7",
    )
    for r in db.search("diabetes", namespace="ward_7",
                       filters={"year": {"gte": 2024}}):
        print(f"{r.score:.2f} {r.signals} {r.text}")
```

Give an assistant memory

```
from vecforge import MemoryStore

mem = MemoryStore("~/.vecforge/memory.db", agent="claude")
mem.remember("User prefers concise answers", kind="preference", importance=0.9)
print(mem.context_pack("how should I answer?"))
```

Or from the terminal

```
vecforge ingest my_docs/ --vault my.db
vecforge search "diabetes" --vault my.db --top-k 5
vecforge memory remember "Deploys happen on Fridays" --kind fact
```

4 Core concepts

Vaults

```
db = VecForge("my_vault.db")                # persistent file
db = VecForge(":memory:")                   # throw-away, for tests
db = VecForge("secure.db", encryption_key=os.environ["VECFORGE_KEY"])
```

Constructor options

Option	Default	Meaning
`path`	`":memory:"`	Vault file
`encryption_key`	None	SQLCipher AES-256 key
`audit_log`	None	JSONL audit log path
`quantum`	False	Use Grover reranking by default
`deletion_protection`	False	Block deletes and replacing upserts
`api_key`	None	Key to check against registered roles (None = local admin)
`model_name`	all-MiniLM-L6-v2	Embedding model, or `"hash"`
`embedder`	None	Your own embedder object
`index_type`	`"auto"`	`flat`, `hnsw`, or `auto` (switches to HNSW at 50,000 documents)
`auto_save_every`	0	Save the index snapshot every N changes
`rerank_model`	ms-marco MiniLM	Cross-encoder used by `rerank=True`

Documents

Every document has an ID (auto UUID or your own), text, a metadata dictionary, a namespace, a modality label (`text`, `image`, `audio`, `memory`) and timestamps.

Namespaces

Namespaces separate tenants, projects or agents inside one vault. A search restricted to a namespace never sees another namespace's documents, and the restriction is applied inside the index so small namespaces get complete results.

```
db.create_namespace("hospital_a")       # also created automatically by add()
db.list_namespaces()
db.delete_namespace("hospital_a")       # removes its documents too
```

Indexes, deletes and snapshots

- **FAISS** holds the vectors. `flat` is exact; `hnsw` is approximate and much faster on very large vaults.
- **BM25** is an inverted index built into VecForge; adding a document only touches that document's words.
- **Deletes** mark the document as gone at once. When deleted entries pass 20% of the vault, the indexes are rebuilt automatically (or call `db.compact()`).
- **Snapshots.** `save()` and `close()` store the indexes inside the vault. On open, VecForge checks the snapshot is current and loads it; if not, it rebuilds from the stored vectors without re-embedding anything.

> **Always close**
>
> Use `with VecForge(...) as db:` or call `db.close()` so the snapshot is saved. If a process is killed, nothing is lost. The next open rebuilds the indexes once.

Thread safety

All public methods of a `VecForge` object take an internal lock, so one vault can be shared by threads, the REST server and a memory store in the same process. Do not open the same file from two processes for writing at the same time.

5 Adding and managing data

Add one document

```
doc_id = db.add(
    "Patient P4821: Type 2 diabetes",
    metadata={"ward": "7", "year": 2026},
    namespace="hospital_a",
    doc_id="P4821",          # optional; error if it already exists
)
```

Add many documents (fastest)

```
ids = db.add_batch(texts, metadata_list=metas, namespace="docs")
```

One model call, one database transaction and one index update for the whole batch. That is 10 to 50 times faster than calling `add()` in a loop. If any custom ID already exists, nothing is inserted.

Insert or replace, and edit metadata

```
db.upsert("Updated discharge note", doc_id="P4821")          # replace text + re-embed
db.update_metadata("P4821", {"reviewed": True})           # merge, no re-embedding
db.update_metadata("P4821", {"only": 1}, merge=False)       # replace
```

Read

```
db.get("P4821")            # SearchResult or None
db.get_embedding("P4821")  # the stored vector
db.exists("P4821")          # also: "P4821" in db
db.count()  /  db.count("hospital_a")  /  len(db)
db.stats()                   # documents, namespaces, index type, model, ...
```

Ingest files and folders

```
count = db.ingest("medical_records/", namespace="records",
                  chunk_size=1000, chunk_overlap=200, batch_size=64)
```

Format	Extensions	Reader
Text, Markdown	`.txt` `.md`	built in
PDF	`.pdf`	PyMuPDF (adds a `page` field)
Word	`.docx`	python-docx

HTML	`.html` `.htm`	BeautifulSoup (scripts, nav and footers removed)

Folders are read recursively. Text is split at paragraph or sentence boundaries; every chunk gets `source`, `filename`, `chunk_index`, `char_start` and `char_end` metadata.

Delete

```
db.delete("P4821")                # True if it existed
db.delete_many(["a", "b", "c"])    # returns how many were deleted
db.delete_namespace("old_tenant")
db.compact()                       # rebuild without deleted entries
```

6 Searching

```
results = db.search(
    "elderly diabetic hip fracture",
    top_k=5,
    alpha=0.7,                          # 70% meaning, 30% keywords
    namespace="ward_7",
    filters={"year": {"gte": 2023}},
    rerank=False, quantum_rerank=False, recency_weight=0.0,
)
```

Choosing alpha

alpha	Behaviour	Good for
0.0	keywords only	IDs, codes, exact terms
0.3	mostly keywords	technical vocabulary
0.5	balanced (default)	general use
0.7	mostly meaning	natural-language questions
1.0	meaning only	concepts, image queries

Metadata filters

Filter	Matches when
`{"type": "NDA"}`	equal
`{"year": {"gte": 2024}}`	also `gt`, `lte`, `lt`
`{"status": {"in": ["a", "b"]}}`	value in list (`not_in` for the opposite)
`{"type": {"ne": "draft"}}`	not equal
`{"tags": {"contains": "ops"}}`	list contains item, or text contains substring
`{"tags": {"any": ["ops", "infra"]}}`	list shares at least one item
`{"tags": {"all": ["ops", "prod"]}}`	list contains all items
`{"reviewed": {"exists": False}}`	key missing or null

Several keys combine with AND. Comparing different types (text against a number) does not match.

Reranking

- `rerank=True`: a cross-encoder reads the query and each candidate together. Most accurate, adds 20 to 50 ms, downloads a second small model on first use.

- `quantum_rerank=True`: Grover amplitude amplification. Candidates found by both the semantic and keyword retrievers move up; the best result always stays first; relative order inside each group is kept. Costs under a millisecond. Use both flags to run the cross-encoder only on the candidates that move up.
- `recency_weight=0.3`: blends in how new each document is, applied after reranking.

Reading results

Field	Meaning
`text`, `doc_id`, `metadata`, `namespace`, `modality`	the document
`score`	relevance, higher is better (0 to 1 unless a cross-encoder is used)
`signals`	`["dense"]`, `["sparse"]` or both, naming the retrievers that found it
`dense_score`, `sparse_score`	per-retriever normalised scores (None if that retriever missed it)
`timestamp`, `updated_at`	created and last edited

More ways to query

```
db.search_batch(["diabetes", "cardiac arrest"], top_k=5)   # one model call
db.similar("P4821", top_k=5)                               # neighbours of a document
db.search("", query_vector=vec, alpha=1.0)                # your own vector
db.search("diabetes", exclude_doc_ids=["P4821"])
```

7 AI memory

`MemoryStore` gives an assistant a memory that survives the end of a conversation. Memories are documents in the namespace `memory:<agent>`, so they share the vault's search, encryption and backups.

```
from vecforge import MemoryStore

mem = MemoryStore("~/.vecforge/memory.db", agent="claude")   # file created on demand
```

Kinds of memory

kind	Store
`preference`	how the user likes things done
`decision`	choices made and why
`fact`	durable facts about the user, project or world
`insight`	conclusions the assistant reached
`task`	open work items
`summary`	session summaries (written by `end_session`)
`episode`	what happened
`note`	anything else (unknown kinds become notes)

Remember

```
m = mem.remember("Deploys happen on Fridays", kind="fact",
                 importance=0.7, tags=["ops"], source="user")
```

If the same thing (or a near-identical sentence of the same kind) is already stored, VecForge reinforces the existing memory instead of adding a copy: its importance and access count rise and tags merge. The returned memory then has `deduplicated=True`. Pass `dedup=False` to force a new entry.

Recall

```
for m in mem.recall("deployment schedule", top_k=5, kinds=["fact", "decision"]):
    print(m.score, m.kind, m.content)
```

The score blends relevance (hybrid search), importance and recency (30-day half-life). Filters: `kinds`, `tags`, `session_id`, `since` (Unix time) and `min_importance`. An empty query returns the most

important recent memories. Each recall updates the access statistics; pass `touch=False` to look without leaving a trace.

Sessions and context packs

```
mem.start_session(metadata={"project": "vecforge"})
mem.remember("Rewrote the reranker", kind="episode")          # linked to the session
mem.end_session("Fixed the reranker and shipped 2.0")       # saved as a summary

# next conversation
mem.start_session()
briefing = mem.context_pack("release", max_chars=4000)
```

`context_pack()` returns Markdown: the last session's summary, then the most relevant memories grouped by kind (preferences and decisions first), trimmed to the character budget (about 4 characters per token). Paste it into a system prompt.

Housekeeping

```
mem.reinforce(memory_id, delta=0.1)
mem.update(memory_id, content="Deploys moved to Thursdays")
mem.forget(memory_id)  ·  mem.forget_many(ids)  ·  mem.clear(kind="episode")
mem.list_memories(kinds=["task"])  ·  mem.list_sessions()  ·  mem.stats()
mem.export_json("memory.json")  ·  other.import_json("memory.json")
```

Several agents in one vault

```
vault = VecForge("~/.vecforge/memory.db", encryption_key=os.environ["VECFORGE_KEY"])
claude = MemoryStore(vault=vault, agent="claude")
coder  = MemoryStore(vault=vault, agent="code-assistant")
```

A store given `vault=` does not close that vault when it closes.

8 Connecting Claude over MCP

The Model Context Protocol lets Claude call tools on your computer. `vecforge mcp` runs a local MCP server so Claude can store and recall memories. Nothing leaves your machine.

Claude Desktop, step by step

1. Install: `pip install "vecforge[mcp]"`
2. Print the configuration: `vecforge mcp --print-config claude-desktop`. Add `--model hash` before `--print-config` for the offline embedder; the printed command keeps it.
3. Open Claude Desktop's config file. On macOS it is `~/Library/Application Support/Claude/claude_desktop_config.json`; on Windows it is `%APPDATA%\Claude\claude_desktop_config.json`. Merge the printed `mcpServers` entry into it.
4. Restart Claude Desktop. The VecForge tools appear in the tools menu.
5. Ask Claude: *"Remember that I prefer short answers."* In a new chat: *"What do you remember about how I like answers?"*

```
{
  "mcpServers": {
    "vecforge-memory": {
      "command": "/usr/bin/python3",
      "args": ["-m", "vecforge.cli.main", "mcp", "--agent", "claude",
               "--vault", "/home/you/.vecforge/memory.db"]
    }
  }
}
```

Claude Code

```
cd your-project
vecforge mcp --print-config claude-code > .mcp.json
```

Claude Code picks up `.mcp.json` in the project root.

HTTP for several clients

```
vecforge mcp --transport http --host 127.0.0.1 --port 8765
```

> **Keep HTTP local**
>
> The HTTP transport has no login. It binds to 127.0.0.1 by default; put an authenticating proxy in front before exposing it to a network.

Tools Claude gets

Tool	What it does
`context_pack`	Briefing of what Claude remembers. Call it at the start of a session
`remember`	Store a memory (duplicates reinforce)
`recall`	Search memories (kinds, tags, since_days, min_importance)
`forget` / `reinforce`	Delete a memory / raise its importance
`list_memories`	Browse without a query
`start_session` / `end_session`	Open a session; close it with a summary for next time
`recent_sessions`	What happened recently
`search_vault` / `add_document`	Search or add ordinary documents in any namespace
`vault_stats`	Statistics

Resources: `vecforge://memory/recent`, `vecforge://memory/sessions`, `vecforge://vault/stats`.
Prompt: `recall_context`.

Options

Option	Default
`--vault`	`~/.vecforge/memory.db`
`--agent`	`claude`
`--transport`	`stdio` (or `http`)
`--model`	all-MiniLM-L6-v2, or `hash`
`VECFORGE_KEY` env var	opens an encrypted vault

Fast sessions

The server loads the embedding model in the background while Claude connects, restores indexes from the snapshot, and saves the snapshot every 25 writes and at shutdown, so the next session starts without a rebuild.

9 Embedding models

An embedder turns text into vectors. VecForge accepts any of these:

Choice	How	Notes
sentence-transformers	`model_name="all-mpnet-base-v2"`	Local, semantic. Default `all-MiniLM-L6-v2` (384 dims, fast).
Hash embedder	`model_name="hash"` or `"hash:512"`	No download, instant, deterministic; lexical only.
Any function or API	`embedder=CallableEmbedder(fn, name="my-model")`	Wrap OpenAI, Voyage, Ollama, ONNX…
CLIP (images + text)	`embedder=ImageEmbedder()`	Needs the multimodal extra.

```
from vecforge import VecForge, CallableEmbedder

def embed(texts):                       # list[str] -> list of vectors
    return my_client.embed(texts)

db = VecForge("api.db", embedder=CallableEmbedder(embed, name="my-model"))
```

Vectors are normalised automatically and repeated texts are cached. The model name is saved with the index; if you open a vault with a different model, VecForge warns you. Re-index everything with:

```
db = VecForge("vault.db", model_name="all-mpnet-base-v2")
db.reembed()
```

You can also supply vectors directly: `db.add("caption", embedding=vec)` and `db.search("", query_vector=vec, alpha=1.0)`.

10 Security

Encryption at rest

```
pip install "vecforge[encryption]"
db = VecForge("secure.db", encryption_key=os.environ["VECFORGE_KEY"])
```

- SQLCipher encrypts the whole file with AES-256: documents, vectors and index snapshots.
- A wrong key fails when the vault opens. A lost key means lost data.
- Without SQLCipher installed, VecForge warns and stores the vault unencrypted.
- Keep keys in environment variables or a secrets manager, never in code.

Roles and API keys

Role	read	write	delete	namespaces	keys	backup
`admin`	✓	✓	✓	✓	✓	✓
`read-write`	✓	✓	✓	×	×	×
`read-only`	✓	×	×	×	×	×
unregistered key	×	×	×	×	×	×

```
admin = VecForge("vault.db")                                  # no key = local admin
admin.register_api_key(os.environ["TEAM_KEY"], "read-only")   # stored as SHA-256 digest
reader = VecForge("vault.db", api_key=os.environ["TEAM_KEY"])
admin.revoke_api_key(os.environ["TEAM_KEY"])
```

`save()` and `compact()` need the write permission. Raw SQLite access through `db.storage` is admin only, because it bypasses every role check.

Audit log

```
db = VecForge("vault.db", audit_log="audit.jsonl")
```

Every add, search, delete and admin action is appended as one JSON line (time, actor, operation, document, namespace, details). Search entries include the query text, so protect the log like the vault. Read it with `AuditLogger("audit.jsonl").read_log(since=…, actor=…, operation=…)`.

Backups

```
db.backup("backups/vault-2026-10-17.db")        # safe while the vault is open
vecforge backup my.db backups/my-copy.db          # same, from the terminal
```

`SnapshotManager` creates timestamped snapshots and restores them, cleaning up stale journal files. Encrypted vaults stay encrypted in backups. `CloudSync` uploads to S3, GCS or Azure but refuses plain `.db` files, so encrypt first.

Deletion protection

`VecForge("vault.db", deletion_protection=True)` blocks `delete`, `delete_many`, `delete_namespace` and upserts that would replace a document.

11 REST API

```
vecforge serve --vault my.db --port 8080                    # http://127.0.0.1:8080/docs
vecforge serve --vault my.db --host 0.0.0.0 --api-key "$VECFORGE_API_KEY"
```

With an API key set, every request needs the header `X-API-Key`. The interactive documentation is at `/docs`.

Method	Path	Purpose
POST	`/api/v1/add`	Add a document
POST	`/api/v1/add_batch`	Add many documents
POST	`/api/v1/search`	Search (query, top_k, alpha, namespace, filters, rerank, quantum_rerank, recency_weight)
GET / PUT	`/api/v1/docs/{id}`	Fetch / insert-or-replace
PATCH	`/api/v1/docs/{id}/metadata`	Update metadata
DELETE	`/api/v1/docs/{id}`	Delete
GET	`/api/v1/docs/{id}/similar`	Similar documents
GET / POST / DELETE	`/api/v1/namespaces`	List / create / delete (`/namespaces/{name}`)
GET	`/api/v1/stats` , `/api/v1/health`	Statistics, health and version
POST	`/api/v1/save` , `/api/v1/compact`	Maintenance
POST	`/api/v1/memory/remember` , `/memory/recall`	Memory per `agent`
GET	`/api/v1/memory/context` , `/memory/sessions`	Briefing, session list
POST	`/api/v1/memory/sessions/start` , `/end`	Sessions
DELETE	`/api/v1/memory/{memory_id}`	Forget

```
curl -X POST http://127.0.0.1:8080/api/v1/search \
  -H "Content-Type: application/json" -H "X-API-Key: $VECFORGE_API_KEY" \
  -d '{"query": "diabetes", "top_k": 5, "quantum_rerank": true}'
```

Status codes: 400 bad input · 401 missing or wrong key · 403 permission or deletion protection · 404 not found or empty vault · 409 duplicate ID.

12 Command line

Every command that opens a vault accepts `--model` (or `VECFORGE_MODEL`).

Command	What it does
`vecforge ingest PATH --vault V`	Ingest files or folders (`--namespace` , `--chunk-size` , `--chunk-overlap`)
`vecforge search "Q" --vault V`	Search (`--top-k --namespace --alpha --rerank --quantum --filters JSON --recency --json`)
`vecforge stats V`	Statistics
`vecforge export V -o out.json`	Export documents as JSON
`vecforge compact V`	Rebuild indexes without deleted entries
`vecforge backup V DEST`	Consistent copy
`vecforge serve --vault V`	REST server (`--host --port --api-key`)
`vecforge mcp`	MCP memory server (`--vault --agent --transport --host --port --print-config`)
`vecforge memory remember "TEXT"`	`--kind --importance --tags`
`vecforge memory recall "Q"`	`--top-k --kinds --json`
`vecforge memory list | sessions | context`	Browse memories, sessions, briefing
`vecforge memory forget ID`	Delete (exit code 1 if not found)
`vecforge memory export FILE | import FILE`	Move memories between machines

```
vecforge search "contract clause" --vault legal.db --top-k 10 \
  --filters '{"type": "NDA", "year": {"gte": 2024}}' --quantum --json
```

13 Multimodal

Install with `pip install "vecforge[multimodal]"`. Whisper also needs `ffmpeg`.

Images

A vault has one vector size. To search images with text, build the vault with CLIP as its embedder so text and images share one space:

```
from vecforge.ingest.vision import ImageEmbedder

clip = ImageEmbedder()
db = VecForge("gallery.db", embedder=clip)
db.add("ward photo", embedding=clip.embed_image("ward.jpg"), modality="image",
       metadata={"path": "ward.jpg"})
db.search("patients in beds", alpha=1.0)                              # text -> image
db.search("", query_vector=clip.embed_image("query.jpg"), alpha=1.0)  # image -> image
```

Audio

```
from vecforge.ingest.audio import AudioEmbedder

audio = AudioEmbedder(text_embedder=db.embedder)
transcript, vec = audio.transcribe_and_embed("meeting.mp3")
db.add(transcript, embedding=vec, modality="audio")
```

Supported audio: mp3, wav, flac, m4a, ogg, webm. Images: jpg, png, webp, bmp, gif, tiff.

14 Performance and tuning

Measured numbers

4-core CPU container, 100,000 documents, hash embedder (so these show database cost, not model cost).

Operation	Result
Hybrid search, exact index	7.3 ms (median)
Search a 500-document namespace	0.8 ms, complete results
Delete	0.1 ms
Ingest (database cost only)	about 4,500 documents per second
Reopen a 5,000-document vault	30 ms
Reopen a 100,000-document vault	1.5 s
Quantum rerank of 1,000,000 candidates	1.7 ms

With a real model, embedding dominates: about 5 ms per text on CPU with MiniLM, far less on a GPU or with batching.

Index types

`index_type`	Search	When
`flat`	exact	up to ~100k documents, or when every result must be exact
`hnsw`	approximate, fast at millions	large vaults; namespace searches under 20k documents are still exact
`auto`	flat, switches to HNSW at 50k	default

Tips

- Use `add_batch` or `ingest`, not `add` in a loop.
- Use `search_batch` for many queries.
- Set `auto_save_every` in long-running services so a crash never forces a rebuild.
- Run `vecforge compact` after very large deletes, and `db.storage.vacuum()` as local admin to shrink the file.
- Put namespaces to work: scoped searches are faster than filtering the whole vault.

15 Upgrading from 1.x

Open your existing vault with 2.0 and it is migrated automatically. The first open rebuilds the indexes once; after that, snapshots make opening fast.

Change	What to do
Unregistered API keys now get no access (1.x gave read-only)	Register keys as admin: `db.register_api_key(key, "read-only")`
`vecforge serve` listens on 127.0.0.1	Pass `--host 0.0.0.0` to expose it, ideally with `--api-key`
`quantum_rerank=True` gives different (correct) orderings, scores from 0 to 1	Re-check any thresholds you tuned on 1.x quantum scores
Dependencies `rank-bm25`, `numba`, `joblib` removed	Nothing; uninstall them if unused
`ImageEmbedder.embed()` on plain text now embeds text	Use `embed_image()` for images explicitly
New result fields (`signals`, `dense_score`, ...)	Optional; existing fields unchanged

Why quantum results changed

VecForge 1.x applied only half of Grover's algorithm (the diffusion step without the oracle). For about half of all result-list sizes that reversed the ranking, and for the rest it did nothing. 2.0 implements the full algorithm and keeps the best result first.

16 Troubleshooting

Symptom	Cause and fix
`VaultEmptyError`	Nothing has been added yet. Add documents first.
`NamespaceNotFoundError`	Typo or the namespace was never created. Check `db.list_namespaces()`.
`VecForgePermissionError`	Your API key is unregistered or its role is too low. Ask an admin to register it.
`DuplicateDocumentError`	That `doc_id` exists. Use `upsert` to replace it.
`VaultClosedError`	The vault was closed. Open it again.
Model download fails / no internet	Use `model_name="hash"` or pre-download the model on a connected machine (Hugging Face cache).
Warning: "SQLCipher is not installed"	`pip install "vecforge[encryption]"`. Until then the vault is not encrypted.
ValueError: "Embedding dimension ... does not match vault dimension"	Vectors you passed come from another model. Nothing was written. Use the vault's model or run `db.reembed()`.
CLI: "File '...' does not exist"	`stats`, `export`, `compact` and `backup` only open existing vaults, so a typo no longer creates an empty file. Check the path.
Warning: "different embedding dimension"	The vault was built with another model. Run `db.reembed()`.
Opening a big vault is slow once	The snapshot was stale (crash or older version). It rebuilds once; close cleanly next time.
Claude does not show VecForge tools	Check the config path and JSON, use the full Python path from `--print-config`, restart Claude. Server logs go to stderr.
`WinError 126 … c10.dll` on Windows	Install the Microsoft Visual C++ Redistributable.
REST returns 401	Send `X-API-Key` matching `--api-key` / `VECFORGE_API_KEY`.

Getting help

Issues: `github.com/bosekarmegam/vecforge/issues` · Contact: `suneelbose@arcgx.in`

17 Quick reference

```
from vecforge import VecForge, MemoryStore, HashEmbedder, CallableEmbedder

# ── open ──
db = VecForge(path, encryption_key=…, model_name=…, index_type="auto", audit_log=…)

# ── write ──
db.add(text, metadata, namespace, doc_id)      db.add_batch(texts, metadata_list, namespace)
db.upsert(text, doc_id)                          db.update_metadata(doc_id, {…}, merge=True)
db.ingest(path, namespace, chunk_size, chunk_overlap)

# ── read ──
db.search(q, top_k, alpha, namespace, filters, rerank, quantum_rerank, recency_weight)
db.search_batch(queries)   db.similar(doc_id)   db.get(doc_id)   db.count()   db.stats()

# ── delete / maintain ──
db.delete(id)  db.delete_many(ids)  db.delete_namespace(ns)  db.compact()
db.save()  db.backup(dest)  db.reembed()  db.close()

# ── security ──
db.register_api_key(key, role)  db.revoke_api_key(key)  db.list_api_keys()

# ── memory ──
mem = MemoryStore(path, agent="claude")
mem.remember(text, kind, importance, tags)    mem.recall(q, top_k, kinds, tags, since)
mem.start_session()  mem.end_session(summary)  mem.context_pack(q, max_chars)
mem.forget(id)  mem.reinforce(id)  mem.update(id, content=…)  mem.export_json(file)
```

Environment variable	Purpose
`VECFORGE_KEY`	Encryption key (used by `vecforge mcp`)
`VECFORGE_MODEL`	Default embedding model (e.g. `hash`)
`VECFORGE_API_KEY`	Key required by the REST server

License

Business Source License 1.1. Free for personal, research, open-source and non-commercial use. Commercial use requires a licence from ArcGX TechLabs Private Limited (suneelbose@arcgx.in).

VecForge is built by Suneel Bose K and powered by ArcGX TechLabs Private Limited.